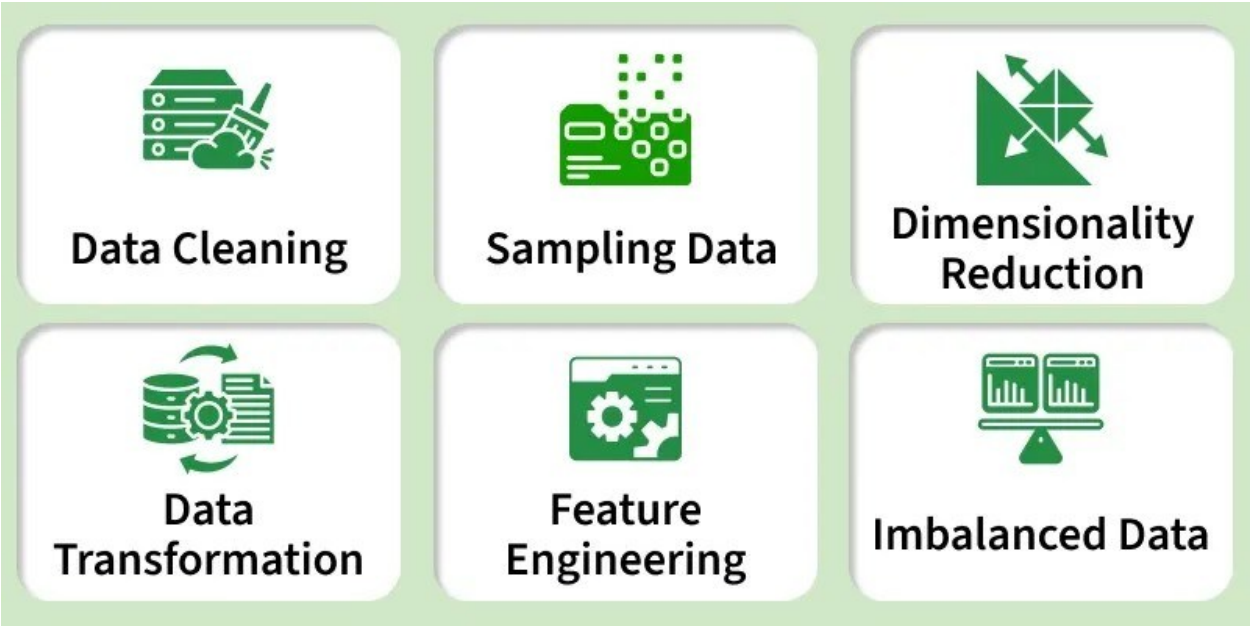
LAB 1 Data mining (Data Preprocessing)

Praproses data adalah langkah pertama dalam setiap alur analisis data atau pembelajaran mesin. Ini melibatkan pembersihan, transformasi, dan pengorganisasian data mentah untuk memastikan data tersebut akurat, konsisten, dan siap untuk dimodelkan. Hal ini memiliki dampak besar pada pembuatan model, seperti:

- Data yang bersih dan terstruktur dengan baik memungkinkan model untuk mempelajari pola yang bermakna, bukan sekadar gangguan.
- Data yang diproses dengan benar mencegah input yang menyesatkan, sehingga menghasilkan prediksi yang lebih andal.
- Data yang terorganisir memudahkan pembuatan input yang bermanfaat untuk model, sehingga meningkatkan kinerja model.
- Data yang terorganisir mendukung Analisis Data Eksplorasi (EDA) yang lebih baik, sehingga pola dan tren menjadi lebih mudah diinterpretasikan.



Implementasi Langkah demi Langkah

Langkah 1: Import Libraries and Load Dataset

Kita menyiapkan lingkungan (environment) dengan pustaka seperti **pandas** , **numpy** , **scikitlearn** , **matplotlib** , **dan seaborn** untuk manipulasi data, operasi numerik, visualisasi, dan penskalaan. Muat dataset untuk pra-pemrosesan.

Dataset sampel dapat diunduh dari sini . (https://drive.google.com/file/d/1F5cRt5LxQ7p_vf1Jq9vmtfAwjlA5hNaQ/view?usp=drive_link)

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('Data/diabetes.csv')
df.head()
```

Output:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Langkah 2: Periksa Struktur Data dan Cari Nilai yang Hilang

Kita memahami ukuran dataset, tipe data, dan mengidentifikasi data yang tidak lengkap (hilang) yang perlu ditangani.

- **df.info():** Mencetak ringkasan singkat termasuk jumlah entri yang tidak null dan tipe data setiap kolom.
- **df.isnull().sum():** Mengembalikan jumlah nilai yang hilang per kolom.

```
df.info()
print(df.isnull().sum())
```

Output:

```
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

Step 3: Statistical Summary and Visualizing Outliers

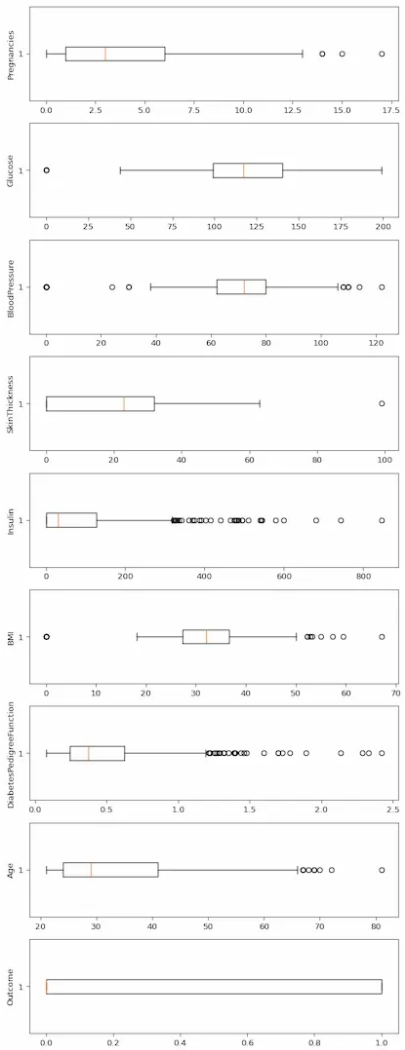
Dapatkan ringkasan numerik seperti rata-rata, median, nilai minimum/maksimum, dan deteksi titik-titik yang tidak biasa (outliers). Pencilan dapat mengacaukan model jika tidak ditangani.

- **df.describe():** Menghitung jumlah, rata-rata, deviasi standar, nilai minimum/maksimum, dan kuartil untuk kolom numerik.
- **Boxplot:** Memvisualisasikan sebaran dan mendeteksi outlier menggunakan fungsi boxplot() dari matplotlib.

```
df.describe()

fig, axs = plt.subplots(len(df.columns), 1, figsize=(7, 18), dpi=95)
for i, col in enumerate(df.columns):
    axs[i].boxplot(df[col], vert=False)
    axs[i].set_ylabel(col)
plt.tight_layout()
plt.show()
```

Output:



Step 4: Remove Outliers Using the Interquartile Range (IQR) Method

Hilangkan nilai ekstrem di luar rentang yang wajar untuk meningkatkan kekokohan model.

- IQR = Q3 (persentil ke-75) – Q1 (persentil ke-25).
- Nilai di bawah Q1 - 1,5IQR atau di atas Q3 + 1,5IQR adalah outlier.
- Hitung batas bawah dan batas atas untuk setiap kolom secara terpisah.
- Saring titik data untuk hanya menyimpan titik data yang berada dalam batas yang ditentukan.

```
q1, q3 = np.percentile(df['Insulin'], [25, 75])
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
clean_df = df[(df['Insulin'] >= lower) & (df['Insulin'] <= upper)]
```

Langkah 5: Analisis Korelasi

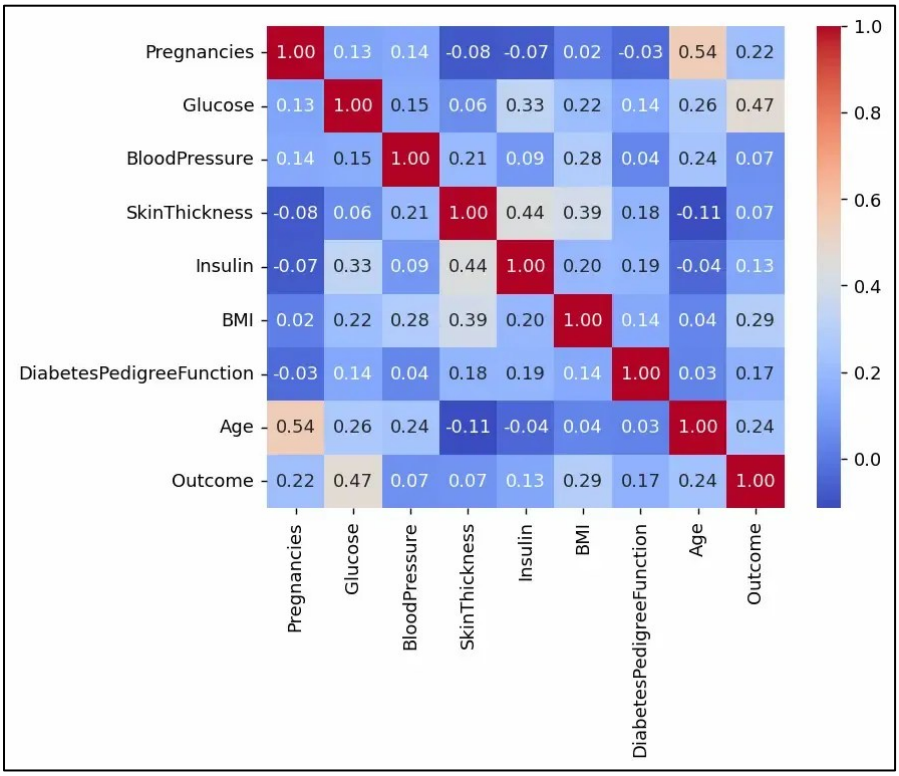
Memahami hubungan antara fitur dan variabel target (Hasil). Korelasi membantu mengukur pentingnya fitur.

- **df.corr():** Menghitung koefisien korelasi berpasangan antar kolom.
- Heatmap melalui seaborn memvisualisasikan matriks korelasi dengan jelas.
- Mengurutkan korelasi dengan corr['Outcome'].sort_values() menyoroti fitur yang paling berkorelasi dengan target.

```
corr = df.corr()
plt.figure(dpi=130)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
```

```
plt.show()

print(corr['Outcome'].sort_values(ascending=False))
```



	Outcome
Outcome	1.000000
Glucose	0.466581
BMI	0.292695
Age	0.238356
Pregnancies	0.221898
DiabetesPedigreeFunction	0.173844
Insulin	0.130548
SkinThickness	0.074752
BloodPressure	0.065068

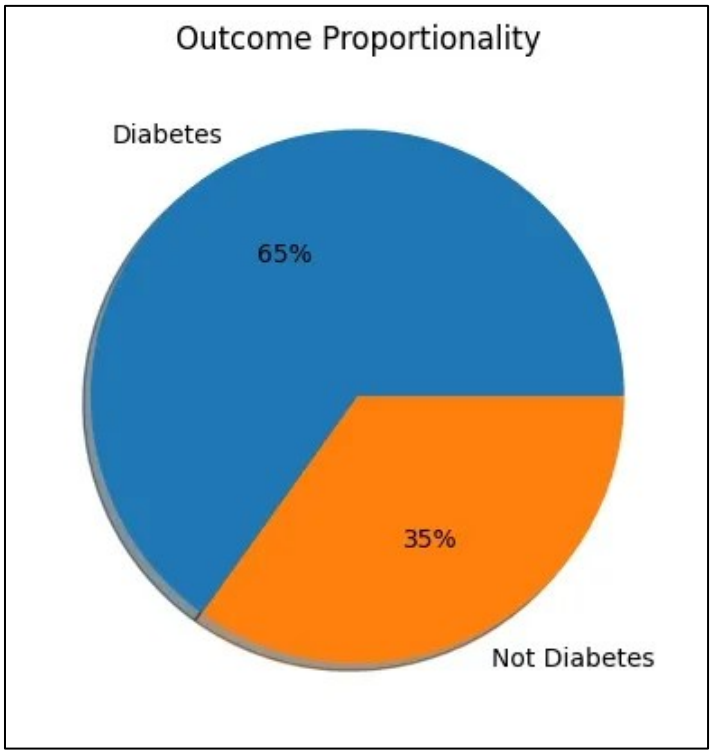
dtype: float64

Langkah 6: Visualisasikan Distribusi Variabel Target

Periksa apakah kelas target (Diabetes vs Bukan Diabetes) seimbang, yang memengaruhi pelatihan dan evaluasi model.

- **plt.pie():** Diagram lingkaran untuk menampilkan proporsi setiap kelas dalam variabel target 'Outcome'.

```
plt.pie(df['Outcome'].value_counts(), labels=[
        'Diabetes', 'Not Diabetes'], autopct='%.f%%', shadow=True)
plt.title('Outcome Proportionality')
plt.show()
```



Langkah 7: Pisahkan Fitur dan Variabel Target

Siapkan variabel independen (fitur) dan variabel dependen (target) secara terpisah untuk pemodelan.

df.drop(columns=[...]): Menghapus kolom target dari fitur.
Pemilihan kolom langsung df['Outcome'] memilih kolom target.

```
X = df.drop(columns=['Outcome'])
y = df['Outcome']
```

Langkah 8: Penskalaan Fitur: Normalisasi dan Standardisasi

Menskalakan fitur ke rentang atau distribusi umum, yang penting untuk banyak algoritma ML yang sensitif terhadap besaran fitur.

1. Normalisasi (Penskalasian Min-Max): Mengubah skala fitur antara 0 dan 1. Baik untuk algoritma seperti k-NN dan jaringan saraf.

- **Kelas:** MinMaxScaler dari sklearn.
- **.fit_transform():** Mempelajari nilai min/max dari data dan menerapkan penskalaan.

```
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
print(X_normalized[:5])
```

Output:

```
array([[0.35294118, 0.74371859, 0.59016393, 0.35353535, 0.        ,
        0.50074516, 0.23441503, 0.48333333],
       [0.05882353, 0.42713568, 0.54098361, 0.29292929, 0.        ,
        0.39642325, 0.11656704, 0.16666667],
       [0.47058824, 0.91959799, 0.52459016, 0.        , 0.        ,
        0.34724292, 0.25362938, 0.18333333],
       [0.05882353, 0.44723618, 0.54098361, 0.23232323, 0.11111111,
        0.41877794, 0.03800171, 0.        ],
       [0.        , 0.68844221, 0.32786885, 0.35353535, 0.19858156,
        0.64232489, 0.94363792, 0.2       ]])
```

2. Standardisasi: Mengubah fitur agar memiliki nilai rata-rata = 0 dan deviasi standar = 1, berguna untuk fitur yang terdistribusi normal.

- **Kelas:** StandardScaler dari sklearn.

```
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)
print(X_standardized[:5])
```

Output:

```
array([[ 0.63994726,  0.84832379,  0.14964075,  0.90726993, -0.69289057,
         0.20401277,  0.46849198,  1.4259954 ],
       [-0.84488505, -1.12339636, -0.16054575,  0.53090156, -0.69289057,
        -0.68442195, -0.36506078, -0.19067191],
       [ 1.23388019,  1.94372388, -0.26394125, -1.28821221, -0.69289057,
        -1.10325546,  0.60439732, -0.10558415],
       [-0.84488505, -0.99820778, -0.16054575,  0.15453319,  0.12330164,
        -0.49404308, -0.92076261, -1.04154944],
       [-1.14185152,  0.5040552 , -1.50468724,  0.90726993,  0.76583594,
         1.4097456 ,  5.4849091 , -0.0204964 ]])
```

Keuntungan

Mari kita lihat keuntungan dari pra-pemrosesan data,

- **Meningkatkan Kualitas Data:** Membersihkan dan mengatur data mentah untuk analisis yang lebih baik.
- **Meningkatkan Akurasi Model:** Menghilangkan noise dan data yang tidak relevan, sehingga menghasilkan prediksi yang lebih tepat.
- **Mengurangi Overfitting:** Menangani outlier dan fitur yang berlebihan, meningkatkan generalisasi model.
- **Mempercepat Pelatihan:** Data yang diskalakan secara efisien mengurangi waktu komputasi.
- **Memastikan Kompatibilitas Algoritma:** Mengonversi data ke dalam format yang sesuai untuk model pembelajaran mesin.